

EPiC Playground

Documentación de Prompts por Módulo

La organización sigue los tres módulos funcionales del sistema:

- EDITOR — Interfaz visual de creación y edición de la red lógica (Frontend React + TypeScript).
 - MOTOR — Cálculo de propagación EPiC sobre tablas de verdad Belnap (Backend Python).
 - SIMULADOR — Orquestación del ciclo de vida de las simulaciones (Backend Python).
-

MÓDULO: EDITOR

Frontend (React + TypeScript) — Edición visual de la red, formularios de nodos/aristas y visualización de resultados.

1. Componente Raíz — EPiC Playground

Archivo	frontend/src/App.tsx
Capa	Frontend / React + TypeScript
Responsabilidad	Componente raíz de la aplicación. Orquesta el canvas SVG interactivo, los formularios de creación y la visualización de resultados.

Prompt

PROMPT
Implementa el componente raíz de EPiC Playground.
Requisitos:
- Editor visual de grafos lógicos con canvas SVG interactivo.
- Formularios para crear nodos (premisa/operador) y aristas.
- Simulación paso a paso con navegación local del historial de propagación.
- Visualización de valores Belnap (T/F/B/N) codificados por colores semánticos.
- Separación clara entre lógica de negocio y capa de renderizado.
- Tipado fuerte con TypeScript.

Requisitos clave

- Separación por capas (lógica / UI).
- Tipado fuerte en TypeScript.
- Preparado para pruebas automatizadas.
- Compatibilidad con la estructura oficial del proyecto.

2. Punto de Entrada React

Archivo	frontend/src/main.tsx
---------	-----------------------

Capa	Frontend / React
Responsabilidad	Monta el componente App dentro de StrictMode en el elemento #root del DOM.

Prompt

PROMPT
Implementa el punto de entrada React del proyecto EPiC Playground.
Requisitos:
- Montar el componente App dentro de React.StrictMode.
- Apuntar al elemento #root del DOM.
- Importar los estilos globales index.css que definen las variables CSS del tema oscuro y el reset de Tailwind.

Requisitos clave

- Usar StrictMode para detección temprana de errores.
- Importar estilos globales antes del montaje.

3. Cliente HTTP del Frontend

Archivo	frontend/src/compartido/api.ts
Capa	Frontend / Compartido
Responsabilidad	Abstrae todas las llamadas REST al backend. Centraliza manejo de errores y configuración de URL base.

Prompt

PROMPT
Implementa el cliente HTTP del frontend para consumir la API REST del backend.
Operaciones requeridas:
- calcularRed()
- iniciarSimulacion()
- obtenerSimulacion()

- pausar / reanudar / avanzar / retroceder paso

Requisitos:

- Usar fetch nativo (sin Axios ni librerías externas).
- Manejo de errores por código HTTP.
- URL base configurable desde la variable de entorno VITE_API_URL.
- Tipado fuerte con TypeScript.
- Desacoplado de los componentes de UI (SRP).

Requisitos clave

- fetch nativo, sin dependencias externas.
- URL base desde variable de entorno VITE_API_URL.
- Manejo explícito de errores HTTP.
- Desacoplado de la UI (SRP).

4. Constantes Visuales del Dominio EPiC

Archivo	frontend/src/compartido/constantes.ts
Capa	Frontend / Compartido
Responsabilidad	Define la paleta de color Belnap, el tema visual oscuro y los presets de redes de ejemplo.

Prompt

PROMPT

Define las constantes visuales del dominio EPiC para uso en toda la aplicación.

Contenido requerido:

- Paleta de color por valor Belnap: T=verde, F=rojo, B=naranja, N=gris.
- Descripciones textuales de cada valor Belnap.
- Colores del tema oscuro del Playground.
- 3 presets de redes de ejemplo: modus-ponens, contradiccion, xor.

Restricciones:

- Solo constantes puras, sin lógica.

- Exportadas como readonly/const para evitar mutaciones accidentales.

Requisitos clave

- Sin lógica, solo constantes puras.
- Exportadas como readonly.
- Reutilizables por cualquier módulo frontend.

5. Definición de Tipos TypeScript — Dominio EPiC

Archivo	frontend/src/compartido/tipos.ts
Capa	Frontend / Compartido
Responsabilidad	Fuente única de verdad de los tipos del sistema. Define enums, interfaces e intersecciones del dominio.

Prompt

PROMPT
Define los tipos TypeScript del dominio EPiC como fuente única de verdad.
Tipos requeridos:
- ValorBelnap: T F B N
- OperadorLogico: AND OR XOR NOT IMP
- Interfaces: Nodo, Arista, Red
- ResultadoPropagacion
- EstadoSimulacion
- NodoConValor: Nodo enriquecido con el valor Belnap actual para renderizado.
Restricciones:
- Solo tipos, sin lógica.
- Compatible con las interfaces del backend (compartido/modelos.py).

Requisitos clave

- Solo declaraciones de tipos, sin lógica de negocio.
- Compatible con modelos.py del backend.
- Exportaciones nombradas para tree-shaking.

6. Canvas SVG Interactivo — Editor de Red

Archivo	frontend/src/editor/CanvasEditor.tsx
Capa	Frontend / Editor
Responsabilidad	Componente visual que renderiza la red EPiC. Maneja interacciones (drag & drop, clic, doble clic) sin contener lógica de negocio.

Prompt

PROMPT
Implementa un canvas SVG interactivo con React para el Editor de redes EPiC.
Renderizado:
- Nodos tipo círculo para operadores lógicos.
- Nodos tipo rombo para premisas.
- Aristas como líneas con punta de flecha.
Interacción:
- Drag & drop de nodos usando refs (evitar stale-closure).
- Eliminar nodos con doble clic / aristas con clic simple.
- Colorear nodos por valor Belnap en modo simulación.
Restricciones:
- Sin lógica de negocio dentro del componente visual (SRP).
- Separar lógica en hooks personalizados.
- Tipado fuerte con TypeScript.

Requisitos clave

- Lógica separada en hooks personalizados (SRP).
- Refs para evitar stale-closure en drag & drop.
- Sin lógica de negocio en el componente visual.
- Tipado fuerte.

7. Formulario de Creación de Aristas

Archivo	frontend/src/editor/FormularioArista.tsx
Capa	Frontend / Editor
Responsabilidad	Formulario React para conectar nodos. Aplica la regla de dominio: las premisas solo pueden ser origen.

Prompt

PROMPT
Implementa un formulario React para crear aristas en la red EPiC.
Campos requeridos:
- Dropdown de origen: cualquier nodo disponible.
- Dropdown de destino: solo nodos de tipo operador.
Reglas de negocio a enforzar en UI:
- Las premisas solo pueden ser origen, nunca destino.
- Para el operador IMP: mostrar ayuda contextual sobre el orden de conexión
(primer input = antecedente, segundo = consecuente).
Restricciones:
- Validaciones en tiempo real antes del submit.
- Mensajes de error claros al usuario.

Requisitos clave

- Validaciones en tiempo real.
- Reglas de dominio reflejadas en la UI.
- Ayuda contextual para operador IMP.
- Mensajes de error descriptivos.

8. Formulario de Creación de Nodos

Archivo	frontend/src/editor/FormularioNodo.tsx
Capa	Frontend / Editor

Responsabilidad	Formulario con toggle premisa/operador. Incluye previsualización de color Belnap e inserción de símbolos lógicos.
------------------------	---

Prompt

PROMPT
Implementa un formulario React con toggle premisa/operador para crear nodos EPiC.
Modo PREMISA:
- Selector de valor Belnap: T / F / B / N.
- Previsualización de color en los botones de selección.
Modo OPERADOR:
- Selector de tipo: AND / OR / XOR / IMP / NOT.
- Descripción del operador seleccionado.
Campo etiqueta:
- Botones de inserción de símbolos lógicos: $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$, $\oplus$.
Restricciones:
- Toggle limpio entre modos sin perder estado del formulario.
- Tipado fuerte.

Requisitos clave

- Toggle limpio entre modo premisa y operador.
- Previsualización de colores Belnap.
- Inserción de símbolos lógicos en etiqueta.
- Tipado fuerte.

9. Controles de Simulación

Archivo	frontend/src/visualizador/ControlesSimulacion.tsx
Capa	Frontend / Visualizador

Responsabilidad	Panel de controles de la simulación: play, pausa, avanzar, retroceder, reiniciar con estado visual.
------------------------	---

Prompt

PROMPT
Implementa el componente de controles de simulación React.
Controles requeridos:
- Botón Simular: dispara la propagación inicial.
- Barra de progreso: iteraciones actual / total.
- Botones: play / pausa / avanzar / retroceder / reiniciar.
- Indicador visual cuando se alcanza punto fijo.
- Indicador visual de error en simulación.
Restricciones:
- Conectarse al estado de simulación vía hooks.
- Sincronizar estado visual con el estado real.
- Permitir extensión futura sin modificar el componente (OCP).

Requisitos clave

- Estado conectado vía hooks (no prop drilling).
- Indicadores visuales de punto fijo y error.
- Extensible sin modificación (OCP).
- Tipado fuerte.

10. Panel de Resultados de Propagación

Archivo	frontend/src/visualizador/PanelResultados.tsx
Capa	Frontend / Visualizador
Responsabilidad	Muestra métricas de propagación, tabla de valores por nodo y leyenda de la lógica EPiC.

Prompt

PROMPT
Implementa el panel de resultados React para EPiC Playground.

Contenido requerido:

- Métricas de propagación: algoritmo usado, número de iteraciones, estado de convergencia.
- Tabla de valores por nodo con indicador de color según Belnap (T/F/B/N).
- Leyenda explicativa de los 4 valores evidenciales de la lógica EPiC.

Restricciones:

- Actualización reactiva ante cambios de estado de simulación.
- Accesible: usar colores + texto, nunca solo color.
- Tipado fuerte.

Requisitos clave

- Actualización reactiva.
- Accesibilidad: color + texto.
- Leyenda Belnap incluida.
- Tipado fuerte.

MÓDULO: MOTOR

Backend (Python/FastAPI) — Cálculo de propagación lógica EPiC mediante tablas de verdad de 4 valores (lógica de Belnap).

1. Paquete Compartido — Backend

Archivo	backend/compartido/__init__.py
Capa	Backend / Python
Responsabilidad	Marca el directorio compartido como paquete Python para habilitar importaciones relativas.

Prompt

PROMPT
Implementa el <code>__init__.py</code> del paquete compartido del backend EPiC.
Objetivo:
- Marcar el directorio como paquete Python.
- Exponer los submódulos: <code>modelos</code> , <code>contratos</code> , <code>validadores</code> , <code>utilidades</code> para facilitar imports en motor y simulador.
Restricciones:
- Mínima lógica; solo exposición de interfaces públicas del paquete.
- Sin dependencias de FastAPI.

Requisitos clave

- Sin lógica de negocio.
- Exponer interfaces públicas del paquete.
- Sin dependencias de FastAPI.

2. Contratos e Interfaces Abstractas

Archivo	backend/compartido/contratos.py
---------	---------------------------------

Capa	Backend / Compartido
Responsabilidad	Define las interfaces abstractas del sistema (DIP). Nadie depende de implementaciones concretas.

Prompt

PROMPT
Define las interfaces abstractas y eventos del dominio EPiC.
Contenido requerido:
- Interfaces: IMotorCalculo, IMotorCalculoIterativo, ISimulacion (usando ABC).
- Enum: FaseSimulacion.
- Eventos como dataclasses: RedActualizada, SimulacionIniciada, ErrorCalculo.
- Excepciones propias del sistema EPiC.
Principios SOLID aplicados:
- DIP: todos los módulos dependen de estas abstracciones, no de implementaciones.
- ISP: interfaces pequeñas y específicas por responsabilidad.
Restricciones:
- Solo abstracciones; sin implementaciones concretas.
- Sin dependencias de FastAPI ni de NumPy.

Requisitos clave

- Solo interfaces y contratos, sin implementaciones.
- DIP: abstracciones como punto de dependencia.
- ISP: interfaces pequeñas y específicas.
- Sin dependencias de frameworks.

3. Entidades del Dominio EPiC

Archivo	backend/compartido/modelos.py
Capa	Backend / Compartido

Responsabilidad	Define las entidades del sistema como dataclasses inmutables con serialización JSON bidireccional.
------------------------	--

Prompt

PROMPT
Define las entidades del dominio EPiC como dataclasses inmutables con serialización JSON.
Entidades requeridas:
- Posicion (x, y)
- Nodo (premisa u operador lógico)
- Arista
- Red
- ResultadoPropagacion
- EstadoSimulacion
- ErrorValidacion
Requisitos:
- Métodos to_dict() / from_dict() bidireccionales en cada entidad.
- Inmutables (frozen=True o equivalente).
- Compatible con los tipos TypeScript del frontend.
- Sin dependencias de FastAPI.

Requisitos clave

- Dataclasses inmutables (frozen=True).
- Serialización bidireccional to_dict / from_dict.
- Compatibles con tipos del frontend.
- Sin dependencias de frameworks.

4. Funciones Auxiliares Puras

Archivo	backend/compartido/utilidades.py
Capa	Backend / Compartido
Responsabilidad	Funciones de utilidad sin dependencias del dominio. Reutilizables por cualquier módulo.

Prompt

PROMPT
Implementa funciones auxiliares puras y sin dependencias del dominio EPiC.
Funciones requeridas:
- generar_id(prefijo): genera IDs únicos con prefijo UUID.
- formatear_decimal(valor): redondeo numérico a 4 decimales.
- clamp(valor, min=0.0, max=1.0): restringe flotante al rango [0, 1].
Restricciones:
- Funciones puras (sin estado ni efectos secundarios).
- Sin imports del dominio EPiC.
- Preparadas para pruebas unitarias directas.

Requisitos clave

- Funciones puras, sin estado.
- Sin imports del dominio.
- Cubiertas por pruebas unitarias.

5. Validadores Estructurales de Red

Archivo	backend/compartido/validadores.py
Capa	Backend / Compartido
Responsabilidad	Validación estructural de redes EPiC. Puede ser usada tanto por backend como por frontend (via contrato).

Prompt

PROMPT
Implementa validación estructural de redes EPiC.
Validaciones requeridas:
- Las premisas no deben recibir entradas.
- NOT debe tener exactamente 1 arista de entrada.

- IMP debe tener exactamente 2 aristas de entrada.
- Todos los IDs deben seguir los formatos: nodo-* / arista-*.
- Los valores de premisa deben pertenecer al conjunto Belnap {T, F, B, N}.
Restricciones:
- Retornar lista de ErrorValidacion, nunca lanzar excepciones directamente.
- Reutilizable por frontend y backend (SRP + ISP).
- Sin dependencias de FastAPI.

Requisitos clave

- Retorna ErrorValidacion[], sin lanzar excepciones.
- Reutilizable por frontend y backend.
- Sin dependencias de FastAPI.
- Cobertura en test_motor.py.

6. Motor de Propagación EPiC — Tablas de Verdad Belnap

Archivo	backend/motor/algoritmos.py
Capa	Backend / Motor
Responsabilidad	Núcleo del sistema. Implementa la propagación lógica sobre la lógica de 4 valores de Belnap.

Prompt

PROMPT
Implementa el motor de propagación EPiC usando tablas de verdad Belnap de 4 valores.
Algoritmo:
- Ordenamiento topológico de la red (algoritmo de Kahn).
- Propagación por punto fijo iterativo (máximo 100 iteraciones).
- Consulta por tipo de operador: NOT, AND, OR, XOR, IMP.
- Leer los valores más recientes en cada pasada.
Tablas de verdad requeridas (4 valores: T, F, B, N):

- NEG (NOT), AND, OR, XOR, IMP.
Salida:
- ResultadoPropagacion con historial completo de pasos.
Restricciones:
- Exclusivamente tablas de verdad; sin álgebra matricial.
- Implementa IMotorCalculo e IMotorCalculoIterativo.
- Sin dependencias de FastAPI.

Requisitos clave

- Implementa IMotorCalculo e IMotorCalculoIterativo (DIP).
- Solo tablas de verdad, sin matrices.
- Historial completo de pasos en la salida.
- Máximo 100 iteraciones para punto fijo.
- Sin dependencias de FastAPI.

7. Endpoints REST del Motor

Archivo	backend/motor/api.py
Capa	Backend / Motor / FastAPI
Responsabilidad	Expone el motor de cálculo como API REST. Valida entrada, ejecuta el motor y cachea resultados.

Prompt

PROMPT
Expone los endpoints FastAPI del módulo Motor.
Endpoints requeridos:
- POST /api/v1/redes/{id}/calcular - recibe red en JSON, ejecuta el motor.
- GET /api/v1/propagaciones/{id} - retorna resultado cacheado.
Requisitos:
- Esquemas Pydantic para validación de entrada.
- Validar estructura de la red antes de ejecutar el motor.

- Cachear resultados en memoria (dict).
- Manejo de errores HTTP apropiados (400, 404, 500).

Restricciones:

- La lógica de cálculo NO debe estar en este archivo (SRP).
- Inyección del motor por fábrica (DIP).

Requisitos clave

- Esquemas Pydantic para validación.
- SRP: sin lógica de cálculo en la capa API.
- DIP: motor inyectado por fábrica.
- Errores HTTP semánticos (400, 404, 500).

8. Pruebas Unitarias del Motor EPiC

Archivo	backend/motor/test_motor.py
Capa	Backend / Motor / Tests
Responsabilidad	Suite completa de pruebas para el motor de propagación.

Prompt

PROMPT
Genera pruebas unitarias completas para el motor EPiC.
Casos a cubrir:
- Tablas de verdad: NEG, AND, OR, XOR (todos los pares de valores Belnap).
- Propagación en cadenas de nodos con premisas T / F / B / N.
- Convergencia a punto fijo.
- Método calcular_pasos: verificar que retorna el historial completo.
Requisitos:
- Framework: pytest.
- Nombres de test descriptivos.
- Casos felices, casos borde y casos inválidos.
- Mocking cuando sea necesario (sin levantar FastAPI).

Requisitos clave

- pytest como framework.
- Cobertura de todas las tablas de verdad.
- Casos felices, borde e inválidos.
- Sin dependencia de FastAPI en las pruebas.

MÓDULO: SIMULADOR

Backend (Python/FastAPI) — Gestión del ciclo de vida de las simulaciones (iniciar, pausar, avanzar, reiniciar) e integración con el Motor.

1. Aplicación FastAPI Principal

Archivo	backend/principal.py
Capa	Backend / FastAPI
Responsabilidad	Crea y configura la aplicación FastAPI. Integra routers de motor y simulador. Punto de entrada del servidor.

Prompt

PROMPT
Crea la aplicación FastAPI principal del sistema ADS (EPiC Playground).
Requisitos:
- Integrar los routers de motor y simulador como submódulos.
- Habilitar CORS para cualquier origen (desarrollo).
- Endpoint GET / con health check y enlace a /docs.
Restricciones:
- Sin lógica de negocio en este archivo.
- Composición de routers, no definición de endpoints de dominio aquí.

Requisitos clave

- Sin lógica de negocio.
- CORS habilitado para desarrollo.
- Health check en GET /.

2. Endpoints REST del Simulador

Archivo	backend/simulador/api.py
---------	--------------------------

Capa	Backend / Simulador / FastAPI
Responsabilidad	Expone el ciclo de vida de la simulación como API REST. Usa Composition Root para inyectar dependencias.

Prompt

PROMPT
Expone los endpoints REST para gestionar simulaciones EPiC paso a paso.
Endpoints requeridos:
- POST /api/v1/simulaciones - iniciar simulación.
- GET /api/v1/simulaciones/{id} - obtener estado actual.
- PUT /api/v1/simulaciones/{id}/pausar
- PUT /api/v1/simulaciones/{id}/reanudar
- PUT /api/v1/simulaciones/{id}/avanzar
- PUT /api/v1/simulaciones/{id}/retroceder
Composition Root:
- Inyectar el motor por fábrica lambda.
- Inyectar GestorEstadoSimulacion en memoria.
Restricciones:
- Sin lógica de simulación en este archivo (SRP).
- Errores HTTP semánticos.

Requisitos clave

- Composition Root: inyección por fábrica (DIP).
- SRP: sin lógica de simulación en la capa API.
- Errores HTTP semánticos.
- Esquemas Pydantic para entrada/salida.

3. Gestor de Estado de Simulaciones

Archivo	backend/simulador/estado.py
Capa	Backend / Simulador

Responsabilidad	Almacena y gestiona el ciclo de vida de cada simulación en memoria. Responsabilidad única: estado.
------------------------	---

Prompt

PROMPT
Implementa GestorEstadoSimulacion como almacén en memoria del ciclo de vida de simulaciones.
Operaciones requeridas:
- crear(red) -> id: genera ID único y registra la simulación.
- obtener(id) -> EstadoSimulacion
- actualizar_fase(id, fase)
- establecer_historial(id, pasos)
- avanzar_paso(id) -> EstadoSimulacion
- retroceder_paso(id) -> EstadoSimulacion
Restricciones:
- Almacenamiento en dict en memoria (sin base de datos).
- Responsabilidad única: solo gestión de estado (SRP).
- Sin lógica de cálculo.

Requisitos clave

- SRP: solo gestión de estado.
- Almacenamiento en memoria (dict).
- Sin lógica de cálculo.
- IDs únicos por simulación.

4. Servicio de Simulación

Archivo	backend/simulador/servicio.py
Capa	Backend / Simulador
Responsabilidad	Orquesta el ciclo completo de una simulación. Recibe el motor por inyección y delega estado al Gestor.

Prompt

PROMPT
Implementa SimulacionServicio cumpliendo la interfaz ISimulacion.
Responsabilidades:
- Recibir el motor por inyección de dependencias vía Callable (DIP).
- Validar la red con validar_red() antes de calcular.
- Ejecutar calcular_pasos() y guardar el historial completo.
- Exponer control de flujo: iniciar, pausar, reanudar, avanzar, retroceder.
Restricciones:
- No acceder directamente al dict de estado; delegar en GestorEstadoSimulacion.
- Sin dependencias de FastAPI.
- Extensible para múltiples estrategias de cálculo (OCP).

Requisitos clave

- DIP: motor recibido por inyección (Callable).
- Valida la red antes de ejecutar.
- Delega estado a GestorEstadoSimulacion.
- Extensible para múltiples motores (OCP).
- Sin dependencias de FastAPI.

5. Pruebas Unitarias del Simulador

Archivo	backend/simulador/test_simulador.py
Capa	Backend / Simulador / Tests
Responsabilidad	Suite completa de pruebas para SimulacionServicio con mocks del motor.

Prompt

PROMPT
Genera pruebas unitarias completas para SimulacionServicio.
Casos a cubrir:
- Inicio exitoso de simulación.

- Verificar que el motor es llamado exactamente una vez.
- Guardado correcto del historial de pasos.
- Navegación de pasos: avanzar y retroceder.
- Pausa y reanudación (cambios de fase).
- Aislamiento entre múltiples simulaciones concurrentes.

Requisitos:

- Framework: pytest + unittest.mock.
- Mockear el motor para no depender de la implementación real.
- Nombres de test descriptivos.
- Casos felices, borde e inválidos.

Requisitos clave

- pytest + unittest.mock.
- Motor mockeado para aislamiento.
- Cobertura de navegación, pausa y concurrencia.
- Sin dependencias de FastAPI.